

Para solucionar el inaceptable consumo de batería, un equipo de ingenieros optimiza las partes críticas del decodificador. Dado que el formato MP3 fue pensado para poder usar aritmética entera, la nueva implementación se escribe en ensamblador utilizando exclusivamente instrucciones enteras (INT) sin usar por tanto la FPU. Esta implementación solo requiere 25 MIPS para decodificar el audio en tiempo real.

d) **Calcula** cuántas veces más durará la batería con el nuevo decodificador (ganancia en consumo).

$F = 25 \text{ M i/s} * 1,2 \text{ c/i} = 30 \text{ MHz}$
 $P = 30 \text{ MHz} * 25 \text{ mw/MHz} = 750 \text{ mW}$
 $G = 5,1 \text{ W} / 0,75 \text{ W} = 6,8 \rightarrow \text{La batería durara 6,8 veces mas!}$

Este procesador tiene direcciones lógicas de 32 bits y direcciones físicas de 24 bits. La jerarquía de memoria tiene las siguientes características.

- Cache L1 tiene un tamaño de 16 Kbytes, es 4-asociativa y se accede con direcciones físicas. La política de escritura es Write through-Write NO Allocate. El tamaño de línea es de 64 bytes.
- TLB de 16 entradas completamente asociativo.
- El tamaño de página del sistema es de 4 KBytes.

e) **Calcula** el número de líneas, vías y conjuntos que tiene la cache L1. **Especifica claramente** cómo has realizado los cálculos.

$16 * 1024 \text{ Bytes} / 64 \text{ Bytes} = 256 \text{ líneas.}$
 4-asociativa $\rightarrow$ 4 líneas por conjunto $\rightarrow$ 4 vías.
 $256 \text{ líneas} / 4 \text{ vías} = 64 \text{ conjuntos}$

La CPU lanza un acceso a la dirección lógica 0x23456789, que el TLB traduce como 0xCDE789.

f) **Indica** en qué conjunto de la L1 se encuentra el dato y cuál es la etiqueta que se guardará en memoria cache L1 (en hexadecimal). **Justifica** la respuesta.

Líneas de 64 bytes $\rightarrow$ 6 bits de byte
 64 conjuntos $\rightarrow$ 6 bits
 @física --- > | TAG (12 bits) | Cjto (6 bits) | byte (6 bits) |
 Cjto = 30 (0x1E en hexa)
 Etiqueta = 0xCDE

Nos gustaría rediseñar la jerarquía de memoria con una cache L1 de 32 KBytes y además acceder en paralelo a la cache L1 y al TLB.

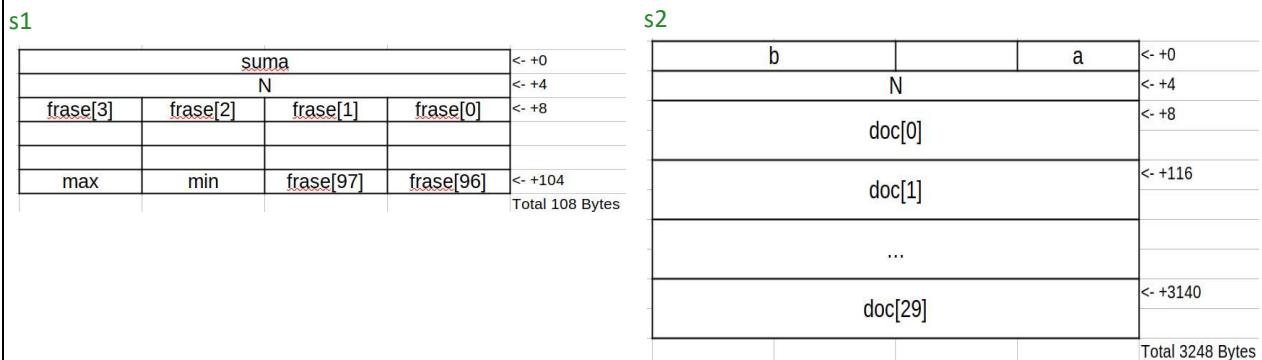
g) **Indica** la mínima asociatividad de la cache que nos permitiría este nuevo diseño **Justifica** la respuesta.

El tamaño máximo de la cache viene delimitado por el grado de asociatividad y por los bits de línea + los bits de conjunto. Estos bits pueden direccionar como máximo una página del sistema (4 KBytes).
 4KBytes (pagina) $\rightarrow$ bits de línea + bits de conjunto = 12 $\rightarrow$ 1 vía = 4KBytes
 Tamaño cache = Tamaño vía * asociatividad = 32Kbytes = 4KBytes*8 $\rightarrow$ 8 asociativa

c) Traduce a ensamblador x86 la función **Troba**. Utiliza preferentemente registros que no hayas de salvar/restaurar.

```
Troba:
    pushl %ebp
    movl %esp, %ebp      # enllac dinamic
    subl $8, %esp        # Reservar VL, es pot eliminar, VL estan en registre
    pushl %ebx           # Guardar nomes ebx.
    movl 8(%ebp), %ecx    # ecx <- x (punter)
    movl $0, %eax        # eax <- max
    movl $0, %edx        # edx <- i
    leal 24(%ebp), %ebx   # ebx <- x2.doc[0] es troba a (ebp+12)+12
for_i:  cmpl 16(%ebp), %edx # x2.N a (ebp+12)+4
        jge fi_for        # salta si x2.N >= 16(%ebp)
        cmpl 4(%ebx), %eax # max <> x2.doc[i].suma
        jge fi_if        # salta si max >= 4(%ebp)
        movb (%ebx), %ah
        movb %ah, (%ecx)  # (*x).min <-- x2.doc[i].min
        movl 4(%ebx), %eax # max = x2.doc[i].suma
fi_if:  incl %edx          # i++
        addl $116, %ebx    # ebx s'incrementa amb tamany s1 .
        jmp for_i
fi_for: popl %ebx
        movl %ebp, %esp    # es pot eliminar si no hem reservat espai per VL
        popl %ebp
        ret
```

d) Reordena los campos de los structs para minimizar el espacio que ocupan en memoria. Dibuja como quedarían almacenadas en memoria las estructuras **s1** y **s2**, indicando los desplazamientos y el tamaño de los structs.



e) ¿Qué dos ineficiencias hay en el paso de parámetros de la función '**Troba**'? En caso de que sea posible, escribe la nueva cabecera de la función '**Troba**' que arregle estos dos problemas.

1. El paràmetre **trobat** no s'usa a la subrutina.
 2. El paràmetre **x2** es passa per valor i usa molt d'espai a la pila
- Solució: `int Troba(s1 *x, s2 *x2)`